

Product Requirements Document (PRD): React Document Builder

1. Project Overview

A web-based React application that allows users to visually design and customize financial documents (receipts, invoices, proforma invoices). The app functions as a template builder where users can define static content, style elements, and insert dynamic placeholders. The final output must be exportable as a single, fully embedded HTML file with inline styles, or a Laravel Blade template ready for backend integration.

2. Target Audience & Use Cases

- **Developers/Designers:** Creating reusable invoice templates for client projects.
- **Business Owners/Agencies:** Designing branded billing documents without writing code.
- **SaaS Integrations:** Generating template structures that a backend (like Laravel) will populate with dynamic database records.

3. Core Features List

3.1. Builder Interface (The Workspace)

- **Canvas Area:** A real-time WYSIWYG (What You See Is What You Get) preview of the document.
- **Block-Based Architecture:** Documents are built by stacking horizontal sections (e.g., Header Banner, Meta Data Grid, Items Table, Total Calculation, Footer).
- **Theme & Global Styling:**
 - Primary and Secondary color pickers (changes table headers, text accents, and background shapes).
 - Typography settings (font family, base size).
 - Document sizing (A4 portrait default).

3.2. Component Library (Drag & Drop Blocks)

- **Graphical Headers & Footers:** Pre-built SVG/CSS shapes (like the slanted/patterned banners in your examples) with customizable colors and background pattern uploads.
- **Logo & Branding:** Image upload block with resizing controls.
- **Information Grids:** 2-column or 3-column layouts for billing info (e.g., "Invoice To:", "Date:", "Invoice No:").
- **Dynamic Data Tables:**
 - Customizable columns (add/remove SL, Description, Price, Qty, Total).
 - Toggleable sub-text (e.g., showing discounted rates under the main description).

- Zebra striping and colored header toggles.
- **Financial Summaries:** Blocks for Subtotal, Tax, Discount, and Grand Total.
- **Static Text Blocks:** For Bank Account Details, Terms & Conditions, and Payment instructions.
- **Icon + Text Rows:** For footer contact info (Phone, Email, Website, Address).

3.3. Variable & Placeholder System

- **Placeholder Insertion:** Users can type or select tokens like `{{ INVOICE_NUMBER }}` or `{{ CLIENT_NAME }}` inside text blocks.
- **Loop Placeholders:** Special wrappers for the Data Table rows (e.g., defining a repeater block for line items).
- **Blade Syntax Support:** Ability to map a placeholder to a Blade variable (e.g., `{{ invoice.number }}` becomes `{{ $invoice->number }}` upon export).

3.4. Export & Save System

- **Export to HTML:** Generates a single .html file. *Crucial:* All CSS must be inlined or contained within a single `<style>` tag, and images must be Base64 encoded so the document has zero external dependencies.
- **Export to Blade:** Generates a .blade.php file. Replaces the builder's visual placeholders with actual Laravel Blade syntax (e.g., `@foreach($items as $item)` for table rows).
- **Save Template (JSON):** Export the raw state of the builder as a .json file so it can be re-uploaded and edited later.

4. Technical Architecture & Stack

To achieve this, you need a stack that supports complex state management and HTML string serialization.

- **Frontend Framework:** React (using Vite for fast compiling).
- **State Management:** **Zustand** or **Redux Toolkit**. You need a centralized store to hold the entire JSON tree of the document's blocks and styles.
- **Drag & Drop:** `@dnd-kit/core` (modern, accessible, and highly customizable for reordering document blocks).
- **Styling (Builder UI):** Tailwind CSS (for the app interface itself).
- **Styling (Document Output):** A CSS-in-JS solution or a custom inline-style parser. *Do not use Tailwind for the exported document*, as most HTML-to-PDF converters (like DomPDF or Snappy) struggle with modern CSS grids/flexbox and external stylesheets. Use standard CSS tables or simple block layouts for the export.
- **HTML Serialization:** `react-dom/server` (`renderToStaticMarkup`). This allows you to take your React components and convert them directly into an HTML string for the export feature.

5. Data Model (State Structure)

Your app's state will need a structured JSON schema. Here is a simplified example of how you should structure the data:

JSON

```
{
  "documentSettings": {
    "type": "invoice",
    "primaryColor": "#6b21a8",
    "fontFamily": "Inter, sans-serif"
  },
  "blocks": [
    {
      "id": "header-123",
      "type": "HeaderBanner",
      "props": { "pattern": "diagonal", "color": "primary" }
    },
    {
      "id": "meta-456",
      "type": "TwoColumnGrid",
      "props": {
        "leftContent": "Invoice To:\n{{ client_name }}\n{{ client_address }}",
        "rightContent": "Invoice No: {{ invoice_no }}\nDate: {{ date }}"
      }
    },
    {
      "id": "table-789",
      "type": "LineItemsTable",
      "props": {
        "columns": ["SN", "Description", "Price", "Qty", "Total"],
        "repeaterVariable": "items"
      }
    }
  ]
}
```

6. Implementation Roadmap

- **Phase 1: Proof of Concept (The Canvas).** Setup React, define the JSON schema, and build a renderer that takes a hardcoded JSON object and displays it visually on screen.
- **Phase 2: The Editor.** Build the sidebar controls. Allow users to add blocks, change text, swap colors, and watch the Canvas update in real-time.
- **Phase 3: The Engine.** Build the HTML serializer. Ensure that clicking "Export" successfully wraps the components in an HTML `<html><body>` boilerplate, inlines the CSS, and downloads the file.
- **Phase 4: Blade Integration.** Add a translation layer to the exporter. When exporting as Blade, parse the text for placeholders and inject `@foreach` and `@if` statements around the table rows and conditional blocks.
- **Phase 5: Polish & Backgrounds.** Implement the complex SVG backgrounds and patterns seen in your examples (the dotted textures and diagonal cuts).

To ensure the architecture perfectly matches your vision, **do you prefer the document builder to be a free-form, drag-and-drop canvas (like Canva, where you can place text absolutely anywhere), or a structured, block-based editor (like Notion or Mailchimp, where sections stack neatly on top of each other)?**